RUNNING TASKS CONDITIONALLY IN ANSIBLE:

- Ansible can use conditionals to execute tasks or plays when certain conditions are met.
- For example, a conditional can be used to determine available memory on a managed host before Ansible installs or configures a service.
- Conditionals allow administrators to differentiate between managed hosts and assign them functional roles based on the conditions that they meet.
- Playbook variables, registered variables, and Ansible facts can all be tested with conditionals. Operators to compare strings, numeric data, and Boolean values are available.

- The following scenarios illustrate the use of conditionals in Ansible:

• A hard limit can be defined in a variable (for example, min_memory) and compared against the available memory on a managed host.

• The output of a command can be captured and evaluated by Ansible to determine whether or not a task completed before taking further action. For example, if a program fails, then a batch is skipped.

• Use Ansible facts to determine the managed host network configuration and decide which template file to send (for example, network bonding or trunking).

• The number of CPUs can be evaluated to determine how to properly tune a web server.

• Compare a registered variable with a predefined variable to determine if a service changed. For example, test the MD5 checksum of a service configuration file to see if the service is changed.

Conditional Task Syntax:

- The `when` statement is used to run a task conditionally.
- It takes as a value the condition to test. If the condition is met, the task runs. If the condition is not met, the task is skipped.

EXAMPLE-1:

```yaml
---
- name: Install packages Based on conditionals
  hosts: rhelnode
  vars:
     my_service: httpd
  tasks:
     - name: "{{ my_service }} package is installed"
       yum:
           name: "{{ my_service }}"
       when:  ansible_distribution == "RedHat"
```

EXAMPLE-2:

```yaml
---
- name: Demonstrate the "in" keyword
  hosts: all
  gather_facts: yes
  vars:
      supported_distros:
         - RedHat
         - Fedora
  tasks:
     - name: Install httpd using yum, where supported
```

```
    yum:
       name: http
       state: present
    when: ansible_distribution in supported_distros
```

Common Distros:

---

ansible_distribution	ansible_os_family
Ubuntu	Debian
RedHat	RedHat
CentOS	RedHat
Fedora	RedHat
Kali	Debian

Examples of Conditionals

OPERATION	Example
Equal (value is a string)	ansible_machine == "x86_64"
Equal (value is numeric)	max_memory == 512
Less than	min_memory < 128
Greater than	min_memory > 256
Less than or equal to	min_memory <= 256
Greater than or equal to	min_memory >= 512
Not equal to	min_memory != 512
Variable exists	min_memory is defined

OPERATION	Example
Variable does not exist	min_memory is not defined
Boolean variable is true (1, True, yes → true)	memory_available
Boolean variable is false (0, False, no → false)	not memory_available
First variable's value is in second variable's list	ansible_distribution in supported_distros

---

Mutilple Conditions

```
when: ansible_distribution == "RedHat" or ansible_distribution == "Fedora"

when: ansible_distribution_version == "7.5" and ansible_kernel == "3.10.0-327.el7.x86_64"
```

- this can be written as:

```
when:

- ansible_distribution_version == "7.5"

- ansible_kernel == "3.10.0-327.el7.x86_64"
```

EXAMPLE:

```yaml
- hosts: all
  become: yes
  vars:
    min_memory: 512   # minimum memory required in MB
  tasks:
    - name: Gather memory facts
      setup:
        filter: ansible_memory_mb


    - name: Install curl if min_memory is defined and sufficient
      apt:
        name: curl
        state: present
      when:
        - min_memory is defined
        - ansible_memory_mb.real.free >= min_memory
```

- **more complex conditions**

```
when: >
( ansible_distribution == "RedHat" and
ansible_distribution_major_version == "7" )
or
( ansible_distribution == "Fedora" and
ansible_distribution_major_version == "28" )
```

You can combine loops and conditionals.

```yaml
- name: Install DB Server
  hosts: rhelnode
  tasks:
    - name: install mariadb-server if enough space on root
      yum:
        name: mariadb-server
        state: latest
      loop: "{{ ansible_mounts }}"
      when: item.mount == "/" and item.size_available > 300000000
```

In the above example, the `mariadb-server` package is installed by the yum module if there is:

a file system mounted on `/` with more than `300 MB` free. The `ansible_mounts` fact is a list of dictionaries, each one representing facts about one mounted file system. The loop iterates over each dictionary in the list, and the conditional statement is not met unless a dictionary is found representing a mounted file system where both conditions are true.